第5章 第11节 LlamaIndex（一）-核心概念与查询机制

- 来源: https://class.imooc.com/lesson/5274#mid=104923
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: LlamaIndex
- 课节: 图文 1-1 LlamaIndex（一）-核心概念与查询机制

1-1 LlamaIndex（一）-核心概念与查询机制

第5章 第11节 LlamaIndex（一）-核心概念与查询机制

> [!TIP] 阅读指南 在前面9节中，我们已经掌握了RAG的核心技术，但你有没有发现一个问题？ 每次构建RAG应用，我们都要重复做这些事： 手动切分文档 自己写检索代码 手动拼接上下文 自己处理各种边界情况 就像在手工作坊里，每个零件都要自己打磨、组装。 有没有一个"自动化工厂"，把这些重复的工作都封装好，专注于业务逻辑？ LlamaIndex，就是为此而生的。

11.1 LlamaIndex是什么

LlamaIndex（原名GPT Index）是一个开源项目，最初是为了解决"如何让LLM访问私有数据"这个问题而产生的。项目迅速获得开发者社区的认可，目前已经发展成为最流行的RAG开发框架之一。它的定位也很清晰——RAG应用的数据框架。

官方定义

> LlamaIndex is a data framework for LLM applications to ingest, structure, and access private or domain-specific data.

翻译过来：LlamaIndex是一个数据框架，帮助LLM应用获取、组织和访问私有或领域数据。

它把RAG开发中的重复性工作都封装好了，只需几行代码就能实现原本需要几百行代码的功能。

LlamaIndex vs 手写RAG

如果不使用任何框架，即使最简单的RAG系统，也颇为繁琐。

但使用LlamaIndex：

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# 1. 加载文档
documents = SimpleDirectoryReader('docs').load_data()

# 2. 构建索引（自动完成分块、向量化、存储）
index = VectorStoreIndex.from_documents(documents)

# 3. 查询
query_engine = index.as_query_engine()
response = query_engine.query("技术文档的问题")

print(response)
```

一切变得非常简洁，因为LlamaIndex做了大量的封装。

LlamaIndex自动完成了：

- 文档分块（默认1024字符，可调整）
- Embedding向量化（默认OpenAI，可切换）
- 向量存储（默认内存，可切换数据库）
- 检索优化（自动Top-K、相似度过滤）
- 上下文拼接（自动组织）
- LLM调用（自动生成答案）

11.2 核心组件架构

> 为了不浪费篇幅，本节只讲LlamaIndex的核心概念和思想。具体API用法和参数请查阅LlamaIndex官方文档。

LlamaIndex的架构很清晰，核心就是把RAG流程分解成4个组件：

```
Document Loaders → Index Structures → Query Engines → Response Synthesizers
   (加载数据)         (构建索引)          (查询接口)        (生成答案)


1
2
```

LlamaIndex核心模块关系图

架构说明

- Settings（配置层）：全局配置中心，统一管理LLM和Embedding模型
- Document Loaders（数据加载层）：支持PDF、Word、网页、数据库等多种数据源
- Index Structures（索引构建层）：将文档组织成可检索的结构（向量索引、关键词索引等）
- Query Engines（查询层）：封装检索逻辑，提供统一查询接口
- Response Synthesizers（生成层）：将检索结果组织成Prompt，调用LLM生成答案

接下来，我们逐一深入讲解每个组件。

11.3 核心组件详解

Settings 统一配置管理

Settings 是全局配置中心，统一设置RAG流程中的两个核心模型：

- 向量编码模型：用于文档向量化和问题向量化
- 大模型：用于生成最终答案

```
from llama_index.core.settings import Settings
from llama_index.llms.dashscope import DashScope
from llama_index.embeddings.dashscope import DashScopeEmbedding

# 全局LLM：对应RAG的「生成阶段」
Settings.llm = DashScope(
    model_name="qwen3-max",
    api_key=API_KEY,
    temperature=0.3
)

# 全局Embedding：对应RAG的「离线阶段」和「检索阶段」
Settings.embed_model = DashScopeEmbedding(
    model_name="text-embedding-v3",
    api_key=API_KEY,
    embed_batch_size=10
)
```

有了Settings，如果不想用Qwen，想换别的模型也很方便，替换这些参数即可。

Document Loaders 数据加载

作用 将各种格式的文档加载成LlamaIndex能处理的Document对象。

作用

支持的格式

```
from llama_index.core import SimpleDirectoryReader
# 1. 本地文件夹（自动识别格式）
documents = SimpleDirectoryReader('./data/docs').load_data()
# 支持：txt, pdf, docx, md, csv, pptx 等

# 2. 单个文件
documents = SimpleDirectoryReader(input_files=['./data/manual.pdf']).load_data()

# 3. 网页
from llama_index.readers.web import SimpleWebPageReader

# 爬虫也给你省了~~
documents = SimpleWebPageReader(html_to_text=True).load_data(
    ["https://example.com/article"]
)

# 4. 数据库
from llama_index.readers.database import DatabaseReader

reader = DatabaseReader(
    connection_string="mysql://user:pass@localhost/dbname"
)
documents = reader.load_data(query="SELECT * FROM articles")
```

不管数据在哪里，LlamaIndex都能将其加载进来。

Index Structures 索引结构

作用 把文档组织成高效的检索结构。

索引的作用

想象有一本1000页的技术手册，用户问了一个问题。如果没有索引，只能从第1页开始一页一页翻，直到找到相关内容——这太慢了！

索引就像书籍的目录和索引页，它提前把文档按照某种规则组织好，实现快速定位到相关内容。

LlamaIndex最常用的是VectorStoreIndex（向量索引），就是我们前面讲的向量检索。它封装了RAG离线阶段的所有工作：文档切块、向量化、存储。

11.4 VectorStoreIndex深入解析

VectorStoreIndex是LlamaIndex最核心的索引类型，本节深入讲解它的分块策略、持久化以及其他索引类型。

分块的重要性

索引的核心是将文档组织成可检索的单元。但整篇文档太大，直接向量化会导致信息损失、检索不精准、上下文窗口溢出。所以，构建索引的第一步就是分块：把大文档切成小块，每个小块单独向量化并索引。这样检索时才能精准定位到相关的小块，而不是整篇文档。

默认分块策略

LlamaIndex默认使用SentenceSplitter进行分块，直接调用from_documents()就会自动切分：

```
from llama_index.core import VectorStoreIndex

# 一行代码完成整个离线阶段
index = VectorStoreIndex.from_documents(documents)
```

内部流程

```
Step 1: 文档切块
        默认1024字符切一块，相邻块重叠20字符
Step 2: 向量化
        调用Settings.embed_model，批量转换为向量
Step 3: 存储
        存入向量库（默认内存，可切换Chroma/Milvus）
```

适用场景 通用文本文档，快速上手。

适用场景

内置Node Parser策略

默认1024字符的分块虽然通用，但并不适合所有场景。比如代码文档应该按函数切分，法律文书应该按段落切分。LlamaIndex提供了多种内置Node Parser，可以根据文档特点灵活选择分块策略，通过transformations参数指定：

```
from llama_index.core.node_parser import (
    SentenceSplitter,      # 按句子切分（默认）
    SemanticSplitterNodeParser,  # 按语义相似度切分
    TokenTextSplitter,     # 按token数切分
    CodeSplitter,          # 按代码结构切分
)

# 示例1：代码文档，按函数/类切分
node_parser = CodeSplitter(
    language="python",
    chunk_lines=40,  # 每块最多40行
)
index = VectorStoreIndex.from_documents(
    code_documents,
    transformations=[node_parser]
)

# 示例2：长文档，按语义自动切分
node_parser = SemanticSplitterNodeParser(
    buffer_size=1,
    breakpoint_percentile_threshold=95  # 语义相似度阈值
)
index = VectorStoreIndex.from_documents(
    long_documents,
    transformations=[node_parser]
)
```

常用Parser

- SentenceSplitter：按句子边界切分，可调整chunk_size
- SemanticSplitterNodeParser：根据语义相似度自动找分割点
- TokenTextSplitter：按token数精确控制
- CodeSplitter：按代码结构（函数、类）切分

完全自定义分块策略

如果需要按特定业务逻辑切分（比如Java规范按条目切分），可以自己分好块再交给LlamaIndex：

```
from llama_index.core import Document, VectorStoreIndex

# 第8节的结构化分块逻辑（按条目切分）
def structured_chunk(text):
    chunks = []
    for item in extract_items(text):  # 提取条目
        chunks.append(item)
    return chunks

# 自己分好块，然后交给LlamaIndex
raw_text = load_pdf("Java开发手册.pdf")
chunks = structured_chunk(raw_text)

# 把每个chunk包装成Document对象
documents = [Document(text=chunk) for chunk in chunks]

# 构建索引（不再自动切分）
index = VectorStoreIndex.from_documents(documents)
```

注意，VectorStoreIndex.from_documents()接收的是Document对象列表。如果已经把文档切好块，每个块包装成一个Document，LlamaIndex就不会再次切分，直接对这些块向量化。

所以，LlamaIndex既可以快速上手（用默认），也可以精细控制（用内置或自定义）。

持久化存储

无论你用哪种分块策略，构建好的索引都可以保存到磁盘，避免每次运行都重新向量化：

```
# 首次运行：构建并保存
index.storage_context.persist(persist_dir="kb/")

# 之后运行：直接加载，不用再次向量化
from llama_index.core import load_index_from_storage, StorageContext
storage_context = StorageContext.from_defaults(persist_dir="kb/")
index = load_index_from_storage(storage_context)
```

除了VectorStoreIndex，LlamaIndex还提供了其他几种索引，它们引出了一个核心要点：RAG ≠ 一定要用向量检索。

RAG的全称是"检索增强生成"（Retrieval-Augmented Generation），关键词是"检索"，而不是"向量检索"。只要能找到相关信息，用什么方式都行：

- 向量检索（Vector Search） ← 最常用 把文本编码成向量，通过余弦相似度查找 优点：语义理解好，能找到"意思相近"的内容 缺点：需要Embedding成本 对应索引 VectorStoreIndex
- 关键词检索（Keyword Search） ← 传统方式 基于BM25、TF-IDF等算法，不需要向量化 优点：快速、精确匹配（比如搜索专业术语） 缺点：不理解语义，"购买"和"采购"会被认为不同词 对应索引 KeywordTableIndex
- 全文遍历（List Search） ← 最简单 不做相似度筛选，按顺序把所有chunk都喂给大模型 优点：信息完整，适合文档量小的总结性任务 缺点：慢、成本高，文档多了会超过上下文窗口 对应索引 ListIndex
- 层次检索（Tree Search） 构建文档摘要树，从根节点向下查找 用LLM判断相关性，逐层筛选 对应索引 TreeIndex

大部分场景用VectorStoreIndex就够了。

11.5 Query Engine与Response Synthesizer

索引构建好之后，就进入查询阶段。LlamaIndex提供了Query Engines和Response Synthesizers两个组件来封装检索和生成逻辑。

Query Engines 查询接口

作用 提供统一的查询接口，封装检索和生成逻辑。对应我们之前RAG流程里的问题相关向量查询这一步。

基础用法

```
# 创建查询引擎
query_engine = index.as_query_engine()

# 查询
response = query_engine.query("你的问题")
print(response)
```

高级配置

```
query_engine = index.as_query_engine(
    similarity_top_k=3,          # Top-K=3，最相关的3条
    response_mode="compact",      # 压缩模式：将多个chunk压缩后一次生成答案
    verbose=True                  # 显示中间过程：输出检索到的chunk和LLM调用详情
)
```

Response Synthesizers 答案合成

作用 决定如何用检索到的chunk调用大模型生成最终答案。

内置策略

```
from llama_index.core.response_synthesizers import get_response_synthesizer

# 1. Refine模式（逐步精炼）
synthesizer = get_response_synthesizer(response_mode="refine")
# 第1个chunk生成初版答案
# 第2个chunk优化答案
# 第3个chunk再优化...

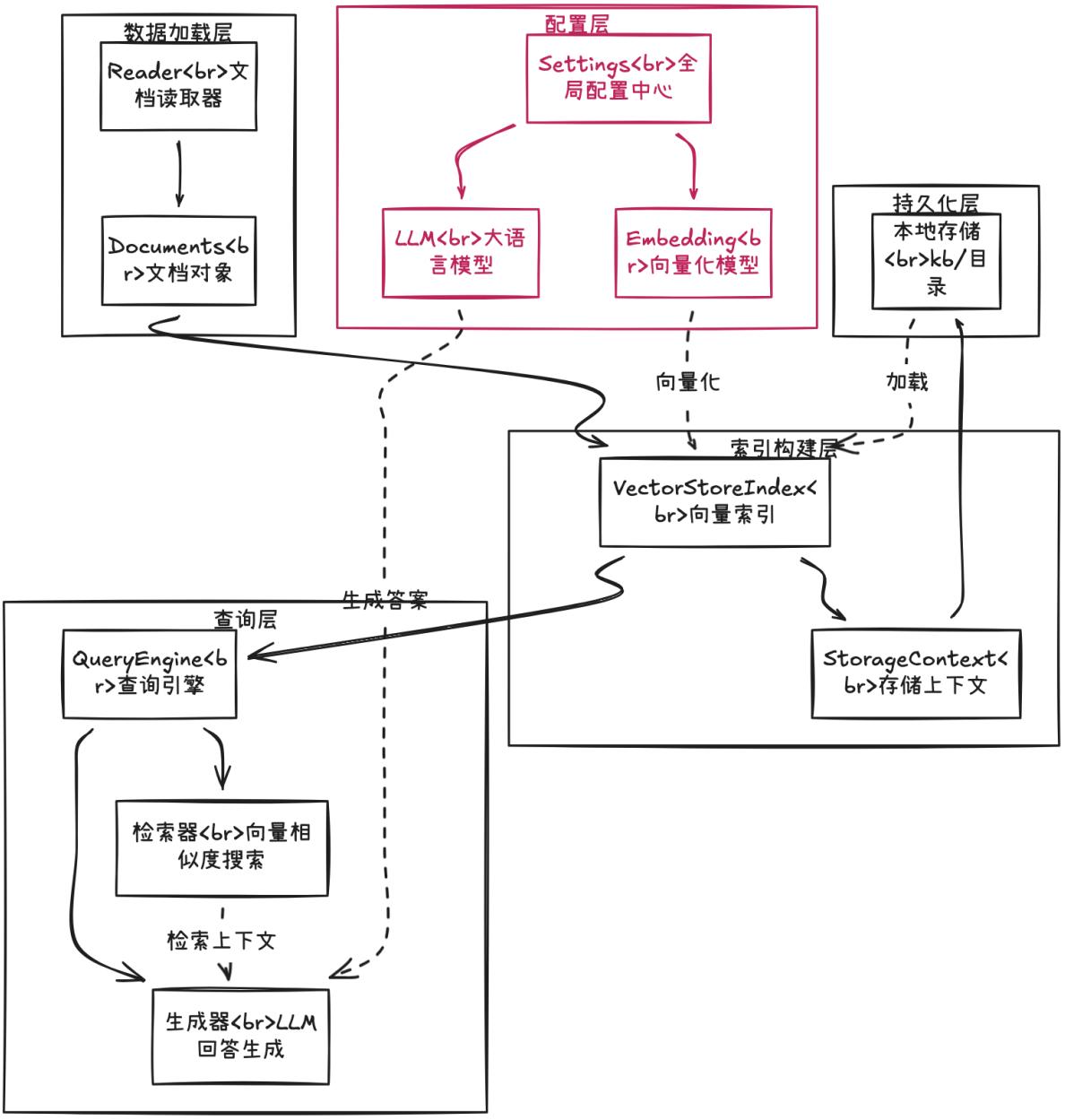
# 2. Tree Summarize（树形摘要）
synthesizer = get_response_synthesizer(response_mode="tree_summarize")
# 把多个chunk分组摘要，再合并

# 3. Compact（压缩拼接）
synthesizer = get_response_synthesizer(response_mode="compact")
# 把所有chunk压缩后一次生成
```

11.6 ■ 学点英语

中文	English	音标	说明
数据框架	Data Framework	/ˈdeɪtə ˈfreɪmwɜːrk/	封装数据处理流程的软件框架，如LlamaIndex
索引构建	Indexing	/ˈɪndeksɪŋ/	将文档解析、分块、向量化后构建可检索索引的过程
检索器	Retriever	/rɪˈtriːvər/	负责从索引中查询相关文档的组件
响应合成器	Response Synthesizer	/rɪˈspɑːns sɪnθəsaɪzər/	将检索结果组装为Prompt并调用LLM生成答案的组件
句子窗口检索	Sentence Window Retrieval	/ˈsentəns ˈwɪndoʊ rɪˈtriːvl/	用小窗口检索 + 大窗口生成的精匹配检索策略

图片


数据加载层
Reader
文档读取器
Documents
文档对象
配置层
Settings
全局配置中心
LLM
大语言模型
Embedding
向量化模型
持久化层
本地存储
kb/目录
向量化
加载
索引构建层
VectorStoreIndex
向量索引
StorageContext
存储上下文
生成答案
查询层
QueryEngine
查询引擎
检索器
向量相似度搜索
检索上下文
生成器
LLM回答生成